SliceKit

VERSION 0.2.1

SliceKit documentation

A Node.js command-line tool for spec-driven agentic delivery, driven from Claude Code through its slash commands.

Contents

1 **Overview** 4

2 **Slash commands** 6

3 **Setup** 8

`init` 8
`update` 9
`workflow` 9

4 **Intake and spec** 10

`branch` 10
`intake snapshot` 10
`intake ask` 11
`intake answer` 11
`intake done` 12
`decide add` 12
`decide settle` 13
`stage require` 14
`spec verify` 14
`gaps scan` 15
`gaps verify` 15
`gaps add` 15
`gaps close` 16
`pr open` 16

5 **Plan** 18

`plan check` 18

6 **Build loop** 19

`build next` 19
`build start` 19
`build parallel` 20
`instructions` 20
`build red` 21
`build commit` 21
`build amend` 22
`build render` 22
`build config` 23
`build pr` 23
`build verdict` 23
`build decline` 24
`build finish` 24
`build sync` 25

7 **Review** 26

`review start` 26

8 **CI and hooks** 27

`check` 27
`ci bookkeeping` 27
`ready-label` 28
`hook` 28

9 The graph and queries 30
graph 30
trace 30
impact 31
history 31
shipped 32
uncovered 32
visualize 32

10 Maintenance 34
status 34
state set 34
state show 35
state unfreeze 36
baseline update 36
baseline show 36
journal verify 37
journal show 37

11 Conventions 38

CHAPTER 1

Overview

What slicekit is

slicekit is a Node.js command-line tool for spec-driven agentic delivery. A requirement document goes in and is snapshotted; an interview turns the answers into numbered decisions; those become a spec in which every requirement cites its source, then a plan of phases and slices. Each slice is then built to a reviewed pull request: failing tests first, then code, a reviewer's verdict and green CI. The rules live in the kit, not in a prompt: each slash command prints its procedure from the CLI, and the CLI, not the agent, runs git.

Reference for the slicekit command-line interface and the slash commands that drive it from Claude Code. Each entry lists the syntax, options, behaviour, when to use it, exit codes and an example.

In your project the kit runs as `npx slicekit`, or through the `cli` value in `.slicekit/config.json`. With a local install that is `node node_modules/slicekit/bin/slicekit.js`. The examples write it as `slicekit`. Run every command from the repository root.

Chapter 2 describes the slash commands. Chapters 3 to 10 describe the CLI commands, grouped by the part of the workflow that uses them, each command in the same order: name, synopsis, description, when to use it, options, exit status and an example. Chapter 11 gathers the conventions every command shares.

The workflow stages

A program moves through these stages in this order. Each stage command refuses to start until the files show the stage before it is complete; `slicekit stage require <stage>` is that check. The stage names in the code are `intake`, `clarify`, `specify`, `spec-review`, `gap-scan`, `stack`, `plan`, `build` and `release`. Merging a pull request is always a person's act.

NO.	STAGE	WHAT HAPPENS, AND THE CLI STEPS IT TAKES
0	**Setup**	Once per repository. Adds the config, docs layout, Claude Code files, git hooks and the CI workflow. `init` · `status --write` · `workflow --write`
1	**Intake** `/slicekit intake <file or URL>`	The source document is snapshotted and frozen. The agent asks one question at a time, and each answer becomes a numbered decision row. `branch spec` · `intake snapshot` · `intake ask` · `intake answer` · `intake done`
2	**Clarify** no command of its own	Questions intake left open are answered. The stage is complete when `decisions.md` exists and the spec cites no blocking Open row. An open row of type `Action` or `Information` is listed as pending and does not block. `decide settle` · `decide add`
3	**Specify, then spec review** `/slicekit specify`	One spec file per capability, every requirement citing its source, a contract per interface. Then the spec pull request; merging it is the spec review. `branch spec` · `stage require clarify` · `spec verify` · `check` · `pr open`

NO.	STAGE	WHAT HAPPENS, AND THE CLI STEPS IT TAKES
4	**Gaps** `/slicekit gaps`	After the spec merges: every place the spec is silent, vague or in conflict with intake becomes a G row, and each row is closed by a decision or a requirement change. `branch gaps` · `stage require spec-review` · `gaps scan` · `instructions auditor` · `gaps add` · `decide settle` · `gaps close` · `gaps verify` · `pr open`
5	**Plan** `/slicekit plan`	The stack, the phases and the slice files, then `BUILD.md` and the plan pull request. `branch plan` · `plan check` · `decide add` · `status --write` · `check` · `pr open`
6	**Build** `/slicekit build [P<n>/S-<nn>]`	One slice: start, failing tests, code until green, a draft pull request, at most two review rounds, green CI. Side by side: `/slicekit build --parallel P<n>` uses `build parallel`. Occasional: `build amend`, `build render`, `build sync`. `stage require plan` · `build next` · `build start` · `instructions developer` · `build red` · `build commit` · `build pr` · `instructions reviewer` · `build verdict` · `instructions fixer` · `build decline` · `build finish`
7	**Independent review, then merge** `/slicekit review <PR>`	In a new Claude Code session with no memory of the build. When it reports `ready`, a person merges. CI then runs `status --sync --write` and the slice becomes `merged`. `review start` · `instructions reviewer` · `build verdict --independent` · `build finish` · `status --sync`
8	**Release** `/slicekit release P<n>`	Specified to write `releases/P<n>.md` and set each slice `released`. **Not built in 0.2.1:** the command prints that it is not built and exits 1. `state set <id> released --release <path>`
9	**The graph and its queries** any time	Build the traceability graph from the docs and git, then ask it questions. `graph` · `trace` · `impact` · `history` · `shipped` · `uncovered` · `visualize`

CHAPTER 2

Slash commands

Each slash command is a short file in `.claude/commands/`. It runs `slicekit instructions --command <name> $ARGUMENTS` and follows the procedure that prints, so the rules live in the kit, not in the prompt. `/slicekit <name> [args]` and `/slicekit-<name> [args]` are the same command. Every procedure runs git only through the CLI, and stops at the first CLI step that exits with a code other than 0.

/slicekit <command> [args]

Runs any of the nine commands below by name, for example `/slicekit build P1/S-02`.

/slicekit-intake <file or URL>

Puts you on `spec/<slug>`, snapshots the source, then interviews you: a product pass (purpose, users, components, existing systems, constraints, out of scope, interfaces), then a platform pass that fills `platform.md` and `observability.md`. One decision row per answer. Ends with the list of questions still open.

When to use. first, with the requirement document. Again for each new source.

/slicekit-specify

Writes `<docsRoot>/spec/<capability>.md` files and a contract per interface from the snapshots and decisions only. Unknowns become `[NEEDS CLARIFICATION: …]` markers. Opens the `[spec]` pull request; merging it approves the spec.

When to use. after intake, once its open questions are settled.

/slicekit-gaps

Scans for gaps, spawns the auditor agent to find more, asks you about each open G row, closes every row, and opens the `[spec] <name>: gaps` pull request.

When to use. after the spec pull request merges, before planning.

/slicekit-plan

Writes `stack.md`, one phase file per milestone with its `**Order:**` line, and the slice files. Regenerates `BUILD.md`, runs `check` until clean, opens the `[plan]` pull request.

When to use. after the gaps pull request merges.

/slicekit-build [P<n>/S-<nn>] [--parallel P<n>]

Builds one slice to a pull request ready to merge: developer agent writes failing tests, then code; reviewer agent gives a verdict; fixer agent answers blockers; at most two rounds; waits for green CI. With no slice it takes the next one. With `--parallel` it starts a whole Order step.

When to use. after the plan pull request merges, once per slice.

`/slicekit-review <PR>`

The independent review: a reviewer in a fresh worktree of the pull request head writes a verdict marked independent. On blockers, it offers a fixer whose commits it pushes to the slice branch. This round does not count against the build's two.

When to use. in a new session, after `/slicekit build` reports `waiting for an independent review`.

`/slicekit-release P<n>`

NOT BUILT IN 0.2.1

Specified (REQ-2.9) to refuse unless every slice of the phase is `merged`, then write `releases/P<n>.md` and set each slice `released`. It does not deploy. Today it prints `the /slicekit release command is not built in this version` and exits 1.

`/slicekit-ops <symptom>`

NOT BUILT IN 0.2.1

Specified (REQ-2.13) to triage a symptom against the runbook and history and return proposed actions, changing nothing. Today it prints that it is not built and exits 1.

`/slicekit-audit [area]`

NOT BUILT IN 0.2.1

Specified (REQ-2.14) to check the spec and runbook claims against the code, tests and CI history. Today it prints that it is not built and exits 1.

CHAPTER 3

Setup

Adding the kit to a repository and keeping its own files current.

---

init

NAME

`slicekit init` — add the kit to a repository

SYNOPSIS

```
slicekit init --shape <backend|fullstack|data> [--name <n>] [--docs <dir>]
              [--cli <command>]
              [--test <cmd>] [--lint <cmd>] [--build <cmd>] [--e2e <cmd>]
```

DESCRIPTION

Adds the kit to the repository: `.slicekit/config.json`, the docs layout, the Claude Code commands, agents and hooks, the git hooks (it sets `core.hooksPath` to `.githooks`) and the CI workflow when there is none yet. It prints `created`, `updated` or `skipped` for each file, then `hooks run: <cli>`.

> **Note.** A new config's `defaultBranch`, which is also its protected branch, is the repository's own: the branch origin's HEAD points at when it still exists; else the first of `main`, `master` and git's `init.defaultBranch` that is a local branch; else, in an empty repository, the branch it is on when it has one of those names; else `main`. An existing config is left as it is.

WHEN TO USE

Once, at the repository root, before anything else. You can run it again; an existing `config.json` wins, so `--shape` is then optional.

OPTIONS

`--shape <backend|fullstack|data>`
: The project's shape; it decides which files are rendered. `fullstack` adds the designer agent. Required unless a config exists.

`--name <n>`
: The program name. Default: the folder's name.

`--docs <dir>`
: The docs root (`docsRoot`). Default: `docs`.

`--cli <command>`
: How the hooks and the workflow run the kit. Default: `node node_modules/slicekit/bin/slicekit.js` with a local install, else `slicekit`. Init warns when the command does not answer `--version`.

`--test, --lint, --build, --e2e <cmd>`
: The team's commands. CI runs one job per declared command; `build pr` runs test, lint and build. No defaults.

EXIT STATUS

0	done (warnings go to stderr)
2	bad usage, not run from the repository root, no `--shape` and no config, or a config problem, including a flag that disagrees with the existing config

EXAMPLE

```
$ npm install --save-dev slicekit
$ npx slicekit init --shape backend --name orders --test "npm test" --lint "npm
run lint"
created .slicekit/config.json
…
hooks run: node node_modules/slicekit/bin/slicekit.js
```

update

NAME — **`slicekit update`** — compare the kit's rendered files with the installed version, and update them

SYNOPSIS — **`slicekit update [--yes] [--check] [--force]`**

DESCRIPTION — Compares every file the kit renders with what the installed version would write. It prints each file that differs, with a status and a diff. Without `--yes` it writes nothing.

WHEN TO USE — After you upgrade the slicekit package. A 0.x version may change how a command works.

OPTIONS

`--yes`
Writes the files whose status is `changed` or `missing`.

`--check`
Writes nothing; exits 1 when any file is `drifted` or `missing`.

`--force`
With `--yes`, also overwrites `drifted` files: kit files someone edited by hand.

EXIT STATUS

0	done, or a dry run
1	with `--check`, a file drifted or is missing; with `--yes`, a drifted file was left in place
2	bad usage or config

EXAMPLE

```
$ npx slicekit update
changed .claude/commands/slicekit-build.md
…
run again with --yes to apply
$ npx slicekit update --yes
```

workflow

CI RUNS --CHECK

NAME — **`slicekit workflow`** — print, write or check the GitHub Actions workflow

SYNOPSIS — **`slicekit workflow [--write | --check]`**

DESCRIPTION — Prints the GitHub Actions workflow the config renders: a gate job, a job per team command, the ready label and the state sync after a merge.

WHEN TO USE — After you change the team commands or CI settings in the config. The generated workflow runs `workflow --check` on every pull request.

OPTIONS

`--write`
Writes `.github/workflows/slicekit.yml`. Warns when the test command never reads `SLICEKIT_SHARD`, since it then runs whole in every shard.

`--check`
Compares that file with the render.

EXIT STATUS

0	printed, written, or matching
1	with `--check`, the file is missing or differs from the render
2	both flags, bad usage or config

EXAMPLE

```
$ slicekit workflow --write
updated .github/workflows/slicekit.yml
```

CHAPTER 4

Intake and spec

The steps of `/slicekit intake`, `/slicekit specify` and `/slicekit gaps`. The agent runs nearly all of them. You type `decide` and `stage require` yourself now and then.

branch

RUN BY THE AGENT

NAME — **`slicekit branch`** — put you on the work branch for a document pull request

SYNOPSIS — `slicekit branch <spec|gaps|plan> <slug>`

DESCRIPTION — Puts you on the work branch `<kind>/<slug>` for a document pull request. A new branch is cut from the default branch as origin has it, after a fetch. A branch that exists here or on origin is taken up where it stands. Untracked files go along; changes to tracked files are refused.

WHEN TO USE — The first step of intake and specify (`spec`), gaps (`gaps`) and plan (`plan`). The slug is the program name, lower-cased, with other characters turned into dashes.

EXIT STATUS

- **0** prints the branch name
- **1** uncommitted changes to tracked files, or a git error
- **2** bad usage, a bad slug (lowercase letters, digits and dashes, at most 60, not starting with a dash), or config

EXAMPLE

```
$ slicekit branch spec orders
spec/orders
```

intake snapshot

RUN BY THE AGENT

NAME — **`slicekit intake snapshot`** — copy a source document into the docs and freeze it

SYNOPSIS — `slicekit intake snapshot <file|url> [--slug <s>] [--who <w>]`

DESCRIPTION — Copies the source document, a file or a URL of at most 50 MB, to `<docsRoot>/intake/YYYY-MM-DD-<slug>.<ext>` beside a sidecar that pins its SHA-256, and commits both. The snapshot is frozen from then on.

WHEN TO USE — Step 2 of `/slicekit intake`. When the source changes, take a new snapshot; never edit the old one.

OPTIONS

`--slug <s>`
: The name part of the file. Default: the source's base name, without extension, made into a slug.

`--who <w>`
: Who supplied the source, written to the sidecar. Default: git `user.name`, else `unknown`.

EXIT STATUS

- **0** prints the snapshot's path
- **1** refused: a protected branch, a failed fetch, a text rule on the target
- **2** bad input: no such file, a bad slug, a URL with a password, a source over the size cap

EXAMPLE

```
$ slicekit intake snapshot ./brief/orders.md
docs/intake/2026-10-31-orders.md
```

intake ask

RUN BY THE AGENT

NAME

slicekit intake ask — record the interview question about to be asked

SYNOPSIS

```
slicekit intake ask --pass <product|platform> --topic <t> --question <q>
```

DESCRIPTION

Records the question the agent is about to ask and prints `question <n>`. It refuses a new question while the last one has no answer row, and a product question once the platform pass has started.

WHEN TO USE

Before every interview question in `/slicekit intake`.

OPTIONS

`--pass <product|platform>`
Which pass the question belongs to. Required.

`--topic <t>`
The topic, such as `users` or a section of `platform.md`. Required.

`--question <q>`
The question as asked. Required.

EXIT STATUS

0	recorded
1	refused
2	bad usage or input

EXAMPLE

```
$ slicekit intake ask --pass product --topic=users --question="Who places an
order?"
question 2
```

intake answer

RUN BY THE AGENT

NAME

slicekit intake answer — write the decision row for the question just asked

SYNOPSIS

```
slicekit intake answer (--answer <a> [--type <T>]
                      | --open --why <w> --recommendation <r>)
                      [--who <w>] [--interface <name> [--source <file>:<line>]]
```

DESCRIPTION

Writes one row to `<docsRoot>/decisions.md` for the question just asked, and prints its ID, such as `D-07`.

WHEN TO USE

After each answer in the interview, before the next `intake ask`.

OPTIONS

`--answer <a>`
The human's answer.

`--type <T>`
`Decision` (default), `Information` for a fact, `Action` for work only a person can do, `Counsel` when an expert view is needed.

`--open`
The human could not answer: the row goes to the Open table. Needs `--why` and `--recommendation`, and takes no `--answer`.

`--why <w>`
With `--open`: why the question matters.

`--recommendation <r>`
With `--open`: what the agent would do.

`--who <w>`
Who answered. Default: git `user.name`, else `unknown`.

`--interface <name>`
Marks the row as naming an interface between components or repositories; `spec verify` then expects a contract for it.

`--source <file>:<line>`
Where the snapshot names that interface. Needs `--interface`.

EXIT STATUS

0 prints the D-ID

1 refused (a file problem)

2 bad usage or input

EXAMPLE

```
$ slicekit intake answer --answer="Shop staff, and customers on the web store"
D-02
```

intake done

RUN BY THE AGENT

NAME — **slicekit intake done** — commit the interview's files and list the open questions

SYNOPSIS

```
slicekit intake done
```

DESCRIPTION — Commits `decisions.md`, `platform.md` and `observability.md`, then prints one `open: <D-ID> <question>` line per open row, or `open: none`. It refuses while a question has no answer, while a pass asked nothing, or when a spec file appeared or changed during the interview.

WHEN TO USE — The last step of `/slicekit intake`.

EXIT STATUS

0 committed

1 refused

2 bad usage

EXAMPLE

```
$ slicekit intake done
open: D-09 Which payment provider do refunds go through?
```

decide add

RUN BY THE AGENT

NAME — **slicekit decide add** — write one decision row outside the interview

SYNOPSIS

```
slicekit decide add (--type <T> --question <q> --answer <a>
                   | --open [--type <T>] --question <q> --why <w>
                     --recommendation <r>)
                   [--who <w>] [--replaces D-<n>]
                   [--interface <name> [--source <file>:<line>]]
```

DESCRIPTION — Writes one decision row to `decisions.md` outside the interview and prints its ID. A settled row is never rewritten: changing your mind is a new row with `--replaces`.

WHEN TO USE

In `/slicekit gaps`, to close a gap with a new answer; in `/slicekit plan`, for each stack choice; any time the team decides something.

OPTIONS

`--type <T>`
One of `Decision`, `Action`, `Information`, `Counsel`, `Open`. Required unless `--open` is given.

`--question <q>`
The question. Required.

`--answer <a>`
The answer, for a settled row.

`--open`
Puts the row in the Open table; needs `--why` and `--recommendation`. `--type Open` does the same.

`--why <w>, --recommendation <r>`
For an open row: why it matters, and what you would do.

`--who <w>`
Who decided. Default: git `user.name`, else `unknown`.

`--replaces D-<n>`
The settled row this one reverses.

`--interface <name>, --source <file>:<line>`
As for `intake answer`.

EXIT STATUS

0	prints the D-ID
1	a file problem (missing file or table, wrong columns)
2	bad usage or input

EXAMPLE

```
$ slicekit decide add --type Decision --question "Which database?" --answer
"Postgres 16"
D-14
```

---

decide settle

RUN BY THE AGENT

NAME

`slicekit decide settle` — move an open decision row to the Settled table

SYNOPSIS

`slicekit decide settle D-<n> --answer <a> [--type <T>] [--who <w>]`

DESCRIPTION

Moves an open row to the Settled table with its answer, keeping the same ID.

WHEN TO USE

The clarify stage: when someone answers a question intake left open. Also in `/slicekit gaps`, since an open row cannot close a gap.

OPTIONS

`--answer <a>`
The answer. Required.

`--type <T>`
Required when the row's Type is `Open`: one of `Decision`, `Action`, `Information`, `Counsel`. Otherwise the row keeps its Type.

`--who <w>`
Who answered. Default: git `user.name`.

EXIT STATUS

- **0** prints the D-ID
- **1** the row is already settled (a reversal is a new row), or a file problem
- **2** an unknown ID, or bad usage or input

EXAMPLE

```
$ slicekit decide settle D-09 --answer "Stripe" --type Decision
D-09
```

stage require

RUN BY THE AGENT

NAME

slicekit stage require — check that a stage and every one before it are complete

SYNOPSIS

slicekit stage require <stage>

DESCRIPTION

Checks from the files that the named stage and every one before it are complete. Stages: `intake`, `clarify`, `specify`, `spec-review`, `gap-scan`, `stack`, `plan`, `build`, `release`. On success it prints `ready: <stage>`; otherwise the first blocker and the command that clears it, on stderr. Pending items that do not block follow.

WHEN TO USE

The gate at the start of specify (`clarify`), gaps (`spec-review`) and build (`plan`). Type it yourself to see where the program stands.

EXIT STATUS

- **0** ready
- **1** a blocker
- **2** bad usage, an unknown stage, or config

EXAMPLE

```
$ slicekit stage require plan
gap scan not done: docs/spec/gaps.md has no Scanned: line; gaps scan has not
run; run /slicekit gaps
```

spec verify

RUN BY THE AGENT

NAME

slicekit spec verify — check that every requirement and interface in the spec is grounded

SYNOPSIS

slicekit spec verify

DESCRIPTION

Checks what the spec stands on: the earlier stages are done, every requirement cites a decision row or intake file that exists, and every interface named at intake has a contract. It lists clarification markers but does not count them as findings.

WHEN TO USE

In `/slicekit specify`, after the spec files are written, until it exits 0.

EXIT STATUS

- **0** prints the counts, then each marker
- **1** one `<file>:<line> <message>` per finding
- **2** bad usage or config

EXAMPLE

```
$ slicekit spec verify
spec verified: 18 REQ, 2 contracts, 1 clarification markers
docs/spec/refunds.md:22 [NEEDS CLARIFICATION: who approves a refund over the
limit?]
```

gaps scan

RUN BY THE AGENT

NAME — **slicekit gaps scan** — add a gap row for each gap a script can find

SYNOPSIS — **slicekit gaps scan**

DESCRIPTION — Adds a row to `<docsRoot>/spec/gaps.md` for each gap a script can find: each "Not yet known" section, each open decision, each requirement that cites one, each interface with no contract. It prints the rows it added, or `no new gaps`.

WHEN TO USE — In `/slicekit gaps`, after the spec pull request merges. Again when `gaps verify` names a gap not in the file.

EXIT STATUS

- **0** scanned
- **1** a file problem
- **2** bad usage

EXAMPLE

```
$ slicekit gaps scan
G-3 unknown D-09 is open: Which payment provider do refunds go through?
```

gaps verify

RUN BY THE AGENT

NAME — **slicekit gaps verify** — check that every gap row is closed and none is missing

SYNOPSIS — **slicekit gaps verify**

DESCRIPTION — Checks the gaps file: every row closed, and no findable gap missing. Prints `ready: gap-scan`, or one line per problem.

WHEN TO USE — In `/slicekit gaps`, after every row is closed and before the pull request.

EXIT STATUS

- **0** ready
- **1** a finding or a file problem
- **2** bad usage

EXAMPLE

```
$ slicekit gaps verify
ready: gap-scan
```

gaps add

RUN BY THE AGENT

NAME — **slicekit gaps add** — write one open gap row

SYNOPSIS — **slicekit gaps add --class <c> --where <file>:<line> --what <text>**

DESCRIPTION — Writes one open gap row and prints its G-ID.

WHEN TO USE — The auditor agent runs it in `/slicekit gaps` for each gap a script cannot find.

OPTIONS

--class <c>
One of `missing`, `unsupported`, `contradiction`, `unknown`, `unrequested`. Required.

--where <file>:<line>
Where the gap is. Required.

`--what <text>`
What is wrong, in one line. Required.

EXIT STATUS

- **0** prints the G-ID
- **1** a file problem
- **2** bad usage or input

EXAMPLE

```
$ slicekit gaps add --class contradiction --where docs/spec/refunds.md:14 --what
"Spec allows 60 days; intake says 30"
G-4
```

gaps close

RUN BY THE AGENT

NAME — **`slicekit gaps close`** — close a gap row by naming what answered it

SYNOPSIS — **`slicekit gaps close G-<n> (--decision D-<n> | --req <REQ>)`**

DESCRIPTION — Closes a gap row by naming what answered it, and prints `G-<n> closed by <ID>`.

WHEN TO USE — In `/slicekit gaps`, after you answer the row's question.

OPTIONS

`--decision D-<n>`
A settled decision row. An open row cannot close a gap.

`--req <REQ>`
A requirement changed on this branch; it must differ from the merged spec.

EXIT STATUS

- **0** closed
- **1** a file problem or a row already closed
- **2** bad usage, bad input, an unknown ID, or not exactly one of the two options

EXAMPLE

```
$ slicekit gaps close G-3 --decision D-09
G-3 closed by D-09
```

pr open

RUN BY THE AGENT

NAME — **`slicekit pr open`** — commit, push and open or update a document pull request

SYNOPSIS

```
slicekit pr open --title <t> --body-file <f> [--commit <message> --paths <a,b>]
                 [--draft] [--base <b>]
```

DESCRIPTION — Optionally commits the named paths, pushes the current branch, and opens its pull request through the git host (the `gh` CLI). If one is already open for the branch, it updates that pull request's body instead. It never pushes a protected branch.

WHEN TO USE — The last step of specify, gaps and plan, with a `[spec]` or `[plan]` title. Slice pull requests use `build pr`.

OPTIONS

`--title <t>`
The pull request title. Required.

`--body-file <f>`
A UTF-8 file holding the body. Required.

`--commit <message>`

Commit message for the paths. Goes with `--paths`.

`--paths <a,b>`

Comma-separated paths to commit. Goes with `--commit`.

`--draft`

Opens the pull request as a draft.

`--base <b>`

The branch it merges into. Default: the default branch. Given, it must match an open pull request's base.

EXIT STATUS

0 prints `#<n> <url>`, or `updated #<n> <url>`

1 git, the host or the body rules refused

2 bad usage, or a body file that cannot be read or is not UTF-8

EXAMPLE

```
$ slicekit pr open --title "[spec] orders" --body-file /tmp/spec-body.md \
    --commit "docs(spec): orders" --paths docs/spec,docs/decisions.md
#12 https://github.com/acme/orders/pull/12
```

CHAPTER 5

Plan

`/slicekit plan` also runs `branch plan`, `decide add`, `status --write`, `check` and `pr open`, described in their own sections.

---

plan check

RUN BY THE AGENT

NAME — `slicekit plan check` — say whether planning may start

SYNOPSIS — `slicekit plan check`

DESCRIPTION — Says whether planning may start. It prints `ready to plan`, or one blocker per line: each clarification marker as `<file>:<line> clarification marker: …`, then the first earlier stage, up to the gap scan, that is not done.

WHEN TO USE — Step 2 of `/slicekit plan`, after the gaps pull request merges.

EXIT STATUS

- **0** ready
- **1** blockers
- **2** bad usage or config

EXAMPLE

```
$ slicekit plan check
ready to plan
```

CHAPTER 6

Build loop

The deterministic steps of `/slicekit build`. Each takes the slice ID, `P<n>/S-<nn>`. A slice's worktree is `.worktrees/P<n>-S-<nn>` on the branch `slice/P<n>-S-<nn>-<descr>`. The agent runs every step; you type one only to resume or inspect by hand.

> **Shared by every `build` subcommand.** `--worktree <p>` names the worktree to act in. Without it the build worktree is used, never the review one, so the building session needs no flag while a review session runs; the review session always passes it. Exit codes: **0** done, **1** refused (the slice's state, git, the git host, the tracker or a held lock), **2** bad usage, an unknown slice ID or a config problem.

build next

RUN BY THE AGENT

NAME — **`slicekit build next`** — print the ID of the next slice to build

SYNOPSIS — **`slicekit build next`**

DESCRIPTION — Prints the ID of the next slice to build: the first `todo` slice whose earlier phases and earlier Order steps are all merged or released, and that no worktree, local branch or branch on origin already holds.

WHEN TO USE — Step 1 of `/slicekit build` when no slice is named.

EXIT STATUS

- **0** prints the ID
- **1** `no slice is ready to build`, then one line per waiting slice

EXAMPLE

```
$ slicekit build next
P1/S-02
```

build start

RUN BY THE AGENT

NAME — **`slicekit build start`** — make and claim a slice's worktree and branch

SYNOPSIS — **`slicekit build start <id> [--descr <d>]`**

DESCRIPTION — Makes the slice's worktree and branch from the default branch, commits an empty claim and pushes it at once, so another clone's `build next` skips the slice. It prints the worktree path, then the branch. For a slice this clone already started it prints `resumed <path> at <state>`, and at `in-review` a second line naming the latest verdict file or the next independent review.

WHEN TO USE — Right after `build next`, or with a slice you name. Also to resume a slice after a crash.

OPTIONS

`--descr <d>`
The description part of the branch name. Default: taken from the slice file name, `S-<nn>-<descr>.md`. `review` is reserved.

EXIT STATUS

- **0** started or resumed
- **1** refused, for example `already started in origin/<branch>; another clone holds it`
- **2** unknown slice, or a branch name git will not accept

EXAMPLE

```
$ slicekit build start P1/S-02
C:/work/orders/.worktrees/P1-S-02
slice/P1-S-02-place-order
```

build parallel

RUN BY THE AGENT

NAME

slicekit build parallel — start every runnable slice of a phase's next Order step

SYNOPSIS

slicekit build parallel P<n>

DESCRIPTION

Starts every runnable slice of the phase's next Order step, each in its own worktree, each claimed as `build start` claims it. It prints `<id> <worktree> <branch>` per slice started, and `slicekit: <id> not started: <reason>` on stderr for the others.

WHEN TO USE

In place of `build next` and `build start` for `/slicekit build --parallel P<n>`.

EXIT STATUS

- **0** every slice that could start did; a slice another clone holds does not count as a failure
- **1** a slice that could have started did not, or none was ready; the started lines are still printed
- **2** the argument is not `P<n>`

EXAMPLE

```
$ slicekit build parallel P1
P1/S-03 C:/work/orders/.worktrees/P1-S-03 slice/P1-S-03-refunds
P1/S-04 C:/work/orders/.worktrees/P1-S-04 slice/P1-S-04-receipts
```

instructions

RUN BY THE AGENT

NAME

slicekit instructions — print an agent's brief or a slash command's procedure

SYNOPSIS

```
slicekit instructions <role> [P<n>/S-<nn>] [--worktree <path> | --summary]
slicekit instructions --command <name> [args...]
```

DESCRIPTION

The first form prints an agent's brief: its rules, the slice file, the cited requirements and decisions, the team's commands and the writing rules. Roles: `developer`, `reviewer`, `fixer`, `auditor`, `ops`, `designer`. Without a slice ID it takes the slice from the current branch. The second form prints a slash command's procedure; everything after the name is passed through unparsed.

WHEN TO USE

Before each agent is spawned: the brief is its whole prompt. Every slash command file runs the `--command` form.

OPTIONS

--worktree <path>
The worktree the agent works in; the brief's Where section names it.

--summary
The short form of the brief, which the subagent-start hook injects. Takes no `--worktree`.

--command <name>
One of `intake`, `specify`, `gaps`, `plan`, `build`, `review`, `release`, `ops`, `audit`. Must come first.

EXIT STATUS

- **0** printed
- **1** the template is not built (`release`, `ops` and `audit` in 0.2.1)
- **2** bad usage, an unknown role or an unknown command name

EXAMPLE

```
$ slicekit instructions developer P1/S-02 --worktree .worktrees/P1-S-02
# Brief: developer
…
$ slicekit instructions --command review 19
```

build red

RUN BY THE AGENT

NAME

slicekit build red — commit the failing tests and freeze them

SYNOPSIS

slicekit build red <id> --tests <a,b> [--worktree <p>]

DESCRIPTION

Runs the test command and requires it to fail: the tests come before the code. Then it commits the tests alone, records them as the slice's frozen tests at `tests-red`, and moves the slice to `building`. It prints the red commit and `tests-red → building`.

WHEN TO USE

After the developer agent reports its test paths, before any code is written.

OPTIONS

--tests <a,b>

The test files, comma-separated, no spaces. Required.

EXIT STATUS

0 recorded

1 refused, for example "the evaluation set must fail before the build"

2 no `--tests`, or an unknown slice

EXAMPLE

```
$ slicekit build red P1/S-02 --tests test/order.test.js,test/order-empty.test.js
3f2a9c1…
tests-red → building
```

build commit

RUN BY THE AGENT

NAME

slicekit build commit — run the tests and commit the worktree's changes

SYNOPSIS

slicekit build commit <id> --message <m> [--require-green] [--worktree <p>]

DESCRIPTION

Runs the tests, then commits the worktree's changes with the test run's result and the slice trailer. It refuses a change to a frozen test or to a kit file. It prints `<sha7> exit <n>`, or `nothing to commit`, which is success.

> **Note.** A machine that cannot run the whole test command at once can set `testCommand` in `.slicekit/local.json`. `build red`, `build commit` and `build pr` then use it and say so. CI never reads that file; never commit it.

WHEN TO USE

Each time a test newly passes; with `--require-green` once all pass; and after the fixer answers review findings.

OPTIONS

--message <m>

The commit subject. Required.

--require-green

Refuses unless every test passes.

EXIT STATUS

0 committed, or nothing to commit

1 a frozen test changed, tests fail under `--require-green`, or the state does not allow it

2 no `--message`

EXAMPLE

```
$ slicekit build commit P1/S-02 --message "feat(P1/S-02): place an order" --require-green
8b41d07 exit 0
```

build amend

RUN BY THE AGENT

NAME

slicekit build amend — unfreeze one frozen test after a person agreed

SYNOPSIS

slicekit build amend <id> --test <path> --reason <r> [--worktree <p>]

DESCRIPTION

Unfreezes one frozen test after a person agreed it may change and the slice file in the worktree was edited to record that. From `draft-pr`, `in-review` or `changes-requested` it also takes the slice back to `building`. It prints `unfroze <test>`.

WHEN TO USE

When the developer reports `amend: <test> — <why>` and you say yes, or when a review finding sits inside a frozen test.

OPTIONS

--test <path>
: The frozen test that may change. Required.

--reason <r>
: Why, recorded in the state history. Required.

EXIT STATUS

0	unfrozen
1	the slice file has no change, the path is not a frozen test, or it is already unfrozen
2	no `--test` or `--reason`

EXAMPLE

```
$ slicekit build amend P1/S-02 --test test/order-empty.test.js --reason "the empty cart returns 422, not 400"
unfroze test/order-empty.test.js
```

build render

RUN BY THE AGENT

NAME

slicekit build render — re-render the kit's own files in a slice that changes its version

SYNOPSIS

slicekit build render <id> [--worktree <p>]

DESCRIPTION

For a slice that changes the kit's own version: runs `update --yes` in the worktree and commits only the files it writes, as `chore(slicekit): <id> render`. It prints `<sha7> render (<n> files)`, or `nothing to render`.

WHEN TO USE

Only when developing slicekit itself, after the green commit. Also when `build commit` says `run slicekit build render <id> again`.

EXIT STATUS

As for every build subcommand.

EXAMPLE

```
$ slicekit build render P2/S-29
c09e2b4 render (12 files)
```

build config

RUN BY THE AGENT

NAME **slicekit build config** — commit a slice's change to the kit's config as the kit's own step

SYNOPSIS **slicekit build config <id> [--worktree <p>]**

DESCRIPTION For a slice whose Expected changes name `.slicekit/config.json`: commits the worktree's changed config alone, as `chore(slicekit): <id> config`. `build commit` never commits a kit file, and accepts this commit only when its subject, trailer and single file match and the slice file names the config. It prints `<sha7> config`, or `nothing to commit`.

WHEN TO USE When a slice changes the team commands, CI settings or another value in the config. New in 0.2.0 (issue #51).

EXIT STATUS As for every build subcommand.

EXAMPLE

```
$ slicekit build config P3/S-32
4be1c07 config
```

build pr

RUN BY THE AGENT

NAME **slicekit build pr** — run the team's commands and the gate, push, and open the draft pull request

SYNOPSIS **slicekit build pr <id> [--notes <file>] [--worktree <p>]**

DESCRIPTION Runs the test, lint and build commands and `slicekit check` in the worktree, pushes, opens the draft pull request (or updates its body), and records `draft-pr`. It prints `#<n> <url>`, then last `verdict <file>`: where the next reviewer writes. A re-run never opens a second pull request. Every refusal comes before the push.

WHEN TO USE When all tests pass, and again after every fix commit.

OPTIONS

`--notes <file>`
: A JSON file for the body: `tryIt` (exactly three steps), `skipped` (one string), and optionally `decisions`, `runbook` and `infra`. Without it the live body is kept; it refuses if that body lacks a section.

EXIT STATUS

0 opened or updated

1 refused: a failing command or check, `pass --notes <file>`, a runbook refusal, or `plan defect:` when another slice of the same Order step changes the same path

2 bad usage or unknown slice

EXAMPLE

```
$ slicekit build pr P1/S-02 --notes /tmp/slicekit-notes-P1-S-02.json
#14 https://github.com/acme/orders/pull/14
verdict C:/work/orders/P1-S-02-r1.md
```

build verdict

RUN BY THE AGENT

NAME **slicekit build verdict** — check, record and push a reviewer's verdict

SYNOPSIS **slicekit build verdict <id> <file> [--independent] [--worktree <p>]**

DESCRIPTION

Checks the reviewer's verdict file, moves it to `.slicekit/reviews/<P-S>-r<k>.md` in the worktree, commits it, records `in-review` and pushes. It prints `next: fix|ask|ci|stuck`, the blockers, the should-haves, `rounds: <used> of 2`, and a new verdict path when another round is allowed.

WHEN TO USE

When the reviewer agent stops. Then act on `next:` — `fix`: fixer; `ask`: ask about should-haves; `ci`: `build finish`.

OPTIONS

`--independent`

Records an independent review's verdict, from the review worktree. It does not count against the two rounds.

EXIT STATUS

- **0** recorded
- **1** `next: stuck` (a blocker survived round 2; the slice is `changes-requested` and the pull request stays a draft), or the verdict was refused
- **2** bad usage or unknown slice

EXAMPLE

```
$ slicekit build verdict P1/S-02 C:/work/orders/P1-S-02-r1.md
next: ask
blockers: none
should-haves: H-1
rounds: 1 of 2
verdict C:/work/orders/P1-S-02-r2.md
```

build decline

RUN BY THE AGENT

NAME

`slicekit build decline` — file a tracker issue for a declined should-have

SYNOPSIS

`slicekit build decline <id> <H-n> [--worktree <p>]`

DESCRIPTION

Files one tracker issue for a should-have you declined, and adds its URL to the pull request body's "Should-haves declined". It prints the URL, followed by `(already filed)` or `(already filed, closed)` when the issue already existed.

WHEN TO USE

After you choose to decline a should-have, in the build or the independent review.

EXIT STATUS

- **0** filed, or already filed
- **1** refused (for example no pull request yet)
- **2** no recorded verdict, or the ID is not a should-have of the latest round

EXAMPLE

```
$ slicekit build decline P1/S-02 H-1
https://github.com/acme/orders/issues/15
```

build finish

RUN BY THE AGENT

NAME

`slicekit build finish` — wait for CI and mark the pull request ready

SYNOPSIS

`slicekit build finish <id> [--wait <s>] [--worktree <p>]`

DESCRIPTION

Waits for CI on the pull request's head. On success it adds the run link to the body's Proof, marks the pull request ready for review, records `ci-green`, and records `ready` when the ready-label decision passes. It prints `ci-green`, the run URL, then `ready`, `ready: waiting for an independent review; …` or `ready: not yet: <reason>`.

WHEN TO USE
On `next: ci`. Run it again while it says CI is still running.

OPTIONS
`--wait <s>`
Seconds to wait for CI, polling every 15 seconds. Default: 90, because a Claude Code Bash call stops at 120 seconds.

EXIT STATUS
0 CI green
1 `CI failed on <sha7>`, `CI still running on <sha7>`, `no CI run found`, or a refusal ending `run slicekit build sync <id>`
2 a `--wait` that is not a whole number, or bad usage

EXAMPLE

```
$ slicekit build finish P1/S-02
ci-green
https://github.com/acme/orders/actions/runs/1187
ready: waiting for an independent review; run /slicekit review 14 in a new
session
```

build sync

RUN BY THE AGENT

NAME
`slicekit build sync` — fast-forward a slice worktree to its branch on origin

SYNOPSIS
`slicekit build sync <id> [--worktree <p>]`

DESCRIPTION
Fast-forwards a slice worktree to its branch on origin and prints the new head.

WHEN TO USE
When another worktree, usually the independent review's, pushed to the slice branch, and a step refused with `run slicekit build sync <id>`. Then run the refused step again.

EXIT STATUS
As for every build subcommand.

EXAMPLE

```
$ slicekit build sync P1/S-02
a71c5e0…
```

CHAPTER 7

Review

`/slicekit review <PR>` starts with `review start`, then uses the build steps with `--worktree <wt>`: `build verdict --independent`, `build decline`, `build commit` and `build pr` for a fix, `build sync` and `build finish`.

---

review start

RUN BY THE AGENT

NAME

`slicekit review start` — check out a pull request's head for an independent review

SYNOPSIS

`slicekit review start <PR>`

DESCRIPTION

Checks out the pull request's head in a fresh worktree, `.worktrees/review-P<n>-S-<nn>`, on `slice/P<n>-S-<nn>-review`, which pushes to the slice branch. It prints four lines: `slice`, `worktree`, `round` and `verdict`. It refuses a pull request that is not open, not a slice pull request or from a fork, and a review worktree with uncommitted changes.

WHEN TO USE

Step 2 of `/slicekit review`, in a new session, after `build finish` says an independent review is waiting. `review` has only this one subcommand.

EXIT STATUS

- **0** ready to review
- **1** refused; if the head `is not origin/<branch>`, try once more after a minute
- **2** the pull request number is not a positive whole number, or config

EXAMPLE

```
$ slicekit review start 14
slice P1/S-02
worktree C:/work/orders/.worktrees/review-P1-S-02
round 2
verdict C:/work/orders/P1-S-02-r2.md
```

CHAPTER 8

CI and hooks

The gate, and the commands the generated workflow and Claude Code's hooks call. You type `check` often; the other three you would rarely type.

check

RUN BY CI · RUN BY CLAUDE CODE

NAME

`slicekit check` — run every rule of the kit's one gate

SYNOPSIS

```
slicekit check [--ci] [--json]
```

DESCRIPTION

The kit's one gate. It runs every rule: `references`, `style`, `queue`, `citations`, `coverage`, `trailer`, `commands`, `hooks`, `journal`, `layout`, `managed` and `workspace`, plus `ci-title` and `ci-body` under `--ci`. It prints each finding, then a summary line such as `check: 0 findings, 0 over baseline, 0 blocking, 14 rules`.

WHEN TO USE

Any time. The Stop hook runs it when the agent tries to end a turn, and blocks the turn while it is red. CI runs `check --ci` on every pull request. `build pr` runs it before every push.

OPTIONS

`--ci`

Adds the pull request title and body rules, reading the pull request from the GitHub event file.

`--json`

Prints one JSON document: findings, counts, over, blocking, exitCode and rules.

EXIT STATUS

0	clean
1	a failing finding
2	cannot run (bad arguments or config)

EXAMPLE

```
$ slicekit check
references docs/spec/refunds.md:9 missing file docs/spec/07-nothing.md
check: 1 findings, 1 over baseline, 1 blocking, 14 rules
```

ci bookkeeping

RUN BY CI

NAME

`slicekit ci bookkeeping` — decide whether a push changes only the kit's bookkeeping

SYNOPSIS

```
slicekit ci bookkeeping [--base <sha>] [--head <sha>] [--base-branch <branch>]
                        [--output <file>]
```

DESCRIPTION

Decides whether the commits of a push change only the kit's own bookkeeping (state files, verdicts, `BUILD.md`). It prints `bookkeeping <base>..<head>`, or `code <base>..<head>`, or `code (<why>)`. The team command jobs are skipped only on `bookkeeping`; any doubt is `code`.

WHEN TO USE

The gate job runs it first on every pull request push. You would type it only to see why CI skipped or ran the tests.

OPTIONS

`--base <sha>`

Compare from this commit. Default: searched for — the newest commit of the pull request, at most 20 back, whose slicekit workflow run passed every team command job and which contains the base branch's tip. An empty or all-zero value means no base, so `code`.

`--head <sha>`

The commit to judge. Default: `HEAD`.

`--base-branch <branch>`

The branch the pull request merges into. Default: the config's default branch.

`--output <file>`

Appends `kind=bookkeeping` or `kind=code` to this file: GitHub's step output.

EXIT STATUS

0 always, whatever the decision

2 bad usage

EXAMPLE

```
$ slicekit ci bookkeeping --head "$SLICEKIT_PR_HEAD" --base-branch main --output
"$GITHUB_OUTPUT"
bookkeeping 8b41d07..a71c5e0
```

ready-label

RUN BY CI

NAME

slicekit ready-label — decide and set a slice pull request's ready label

SYNOPSIS

slicekit ready-label

DESCRIPTION

Decides whether a slice pull request gets the `ready` label, and with `GH_TOKEN` set, adds or removes it (creating the label if needed). Ready means: the title names a slice, `SLICEKIT_CHECKS_OK=true`, the latest verdict on the current head has no open blockers, and so does the latest independent verdict, unless `review.independent` on the base branch makes it optional. It prints the decision and its reason. The label is information; merging stays a person's act.

WHEN TO USE

The workflow's ready-label job, after the other jobs. It reads the pull request from `GITHUB_EVENT_PATH` and the base from `GITHUB_BASE_REF`.

EXIT STATUS

0 always, even on bad usage or a failed label call (that becomes a GitHub warning)

EXAMPLE

```
$ SLICEKIT_CHECKS_OK=true slicekit ready-label
ready-label: P1/S-02 not ready: independent review missing
ready-label: GH_TOKEN not set; label left unchanged
```

hook

RUN BY CLAUDE CODE

NAME

slicekit hook — run one Claude Code hook handler

SYNOPSIS

slicekit hook <name>

DESCRIPTION

Runs one hook handler: reads a JSON object on stdin and prints one JSON decision. Names: `session-start`, `pre-bash`, `pre-edit`, `subagent-start`, `subagent-stop`, `pre-compact`, `stop`, `session-end`. The first, `subagent-start`, `pre-compact` and `session-end` fail open: on an error they answer `{}`. The others fail closed: they deny the tool call or block the stop. Outside a slicekit project every hook answers `{}`.

WHEN TO USE

Never by hand in normal use. `init` writes these into `.claude/settings.json` for SessionStart, PreToolUse (Bash; Edit, Write, MultiEdit and NotebookEdit), SubagentStart, SubagentStop, PreCompact, Stop and SessionEnd. Pipe JSON into it to test a guard.

EXIT STATUS

0	for every known name; the decision is in the JSON
2	no name, or an unknown name

EXAMPLE

```
$ echo '{"tool_name":"Bash","tool_input":{"command":"git push --force origin
main"},"cwd":"."}' | slicekit hook pre-bash
{"hookSpecificOutput":
{"hookEventName":"PreToolUse","permissionDecision":"deny","permissionDecisionRea
son":"push to protected branch main"}}
```

CHAPTER 9

The graph and queries

`graph` builds `.slicekit/graph/graph.json` from the kit's IDs and from git: requirements, decisions, slices, tests, commits, pull requests and releases. It is not a code graph. The queries read only that file; they run no git and no host call.

> **Shared by the five queries** (`trace`, `impact`, `history`, `shipped`, `uncovered`). `--json` prints one JSON document, also on an error (`{"error": …}`). Exit codes: **0** answered, **1** no graph (`run slicekit graph`), a graph of an older schema, or a phase with no release record, **2** bad usage, an ID or path not in the graph, or config.

graph

CI RUNS IT TOO

NAME

`slicekit graph` — build the traceability graph

SYNOPSIS

```
slicekit graph [--full]
```

DESCRIPTION

Builds the traceability graph. It is incremental: it walks only new commits and reads only changed files. It is the only command that reads pull requests from the git host; without a host it skips them with a note. It also walks the repositories of a multi-repo workspace. It prints the graph's path, the node and edge counts, what it walked and read, and each repository it could not reach.

WHEN TO USE

Before any query, after merges. The generated workflow builds it on every pull request and keeps it as an artifact.

OPTIONS

`--full`
Rebuilds from scratch, ignoring the saved index.

EXIT STATUS

0	written
1	a schema problem, the default branch not present, or a bad workspace file
2	not a git repository, bad usage or config

EXAMPLE

```
$ slicekit graph
.slicekit/graph/graph.json
nodes 1240 edges 3315
commits walked 12 of 412
files read 9 of 188
pull requests fetched 2, from the cache 31
```

trace

NAME

`slicekit trace` — trace a requirement down, or a file up

SYNOPSIS

```
slicekit trace <REQ-…> [--json]
slicekit trace --file <path> [--json]
```

DESCRIPTION

For a requirement: the tree under it — its slices with their pull requests, commits and files, then the tests that cover it. For a file: the tree above it — the slices whose commits touched it, with the requirements and decisions above each.

WHEN TO USE

"What delivered this requirement?" or "why does this file exist?"

OPTIONS

`--file <path>`

Trace upward from a file instead of down from a requirement.

EXIT STATUS

As for every query. Give exactly one requirement or `--file`.

EXAMPLE

```
$ slicekit trace REQ-2.8
requirement REQ-2.8 `/slicekit review <PR>`
  slice P2/S-16 `/slicekit review` and `--parallel` (merged)
    pull request #19 https://github.com/adityapandya1/agentic-slicekit/pull/19
    commit af5e0d7 ci: the kit's own jobs get 60 minutes
      file .github/workflows/slicekit.yml
…
```

---

impact

NAME

`slicekit impact` — list what shares files or contracts with a target

SYNOPSIS

`slicekit impact <REQ-…|C-…|P<n>/S-<nn>|path> [--json]`

DESCRIPTION

The contracts, slices, files and repositories that share files or contracts with the target: a requirement, a contract, a slice or a path (a directory matches every file under it). The kit's bookkeeping files never count, since every slice touches them.

WHEN TO USE

Before changing something: "what else could this break?"

EXIT STATUS

As for every query.

EXAMPLE

```
$ slicekit impact src/orders/
contracts:
  C-1 payments-api
slices:
  P1/S-02  repo .  touched 4 shared files
files:
  src/orders/place.js
…
```

---

history

NAME

`slicekit history` — list the slices whose commits touched the paths

SYNOPSIS

`slicekit history <path> [<path>…] [--json]`

DESCRIPTION

Newest first, the slices whose commits touched the paths, each with its pull request, its last verdict and its release. Commits in no slice are not listed.

WHEN TO USE

"Who changed this, under which slice, and was it reviewed?"

EXIT STATUS

As for every query. Give at least one path.

EXAMPLE

```
$ slicekit history src/orders/place.js
P1/S-02  PR #14  APPROVE  unreleased
```

shipped

NAME — **`slicekit shipped`** — print a phase's release record

SYNOPSIS — **`slicekit shipped P<n> [--json]`**

DESCRIPTION — A phase's release record: its path, the requirements it delivered and its pull requests with their URLs.

WHEN TO USE — After a release. Release records come from `/slicekit release`, which is not built in 0.2.1, so today it answers `P<n> has no release record` unless one was written by hand.

EXIT STATUS — As for every query; **1** for a phase with no release record.

EXAMPLE

```
$ slicekit shipped P1
slicekit: P1 has no release record
```

uncovered

NAME — **`slicekit uncovered`** — list untested requirements, unlinked tests and commits outside a slice

SYNOPSIS — **`slicekit uncovered [--json]`**

DESCRIPTION — Three lists: requirements no test covers, test files that name no requirement, and commits on the default branch with no `Slice:` trailer (merge commits left out).

WHEN TO USE — Before a release, or to find work done outside the loop.

EXIT STATUS — As for every query. It takes no argument.

EXAMPLE

```
$ slicekit uncovered
requirements with no covering test: 1
  REQ-3.2 …
tests naming no requirement: 0
commits on main with no slice trailer: 2
…
```

visualize

NAME — **`slicekit visualize`** — show the graph as one self-contained page

SYNOPSIS — **`slicekit visualize [--port <n>] [--write <file>]`**

DESCRIPTION — Shows the graph as one self-contained page. By default it serves the page and `graph.json` on 127.0.0.1 only, prints the URL, opens nothing, and runs until Ctrl-C. With `--write` it writes the page to a file and prints its path.

WHEN TO USE — To browse the graph, or to attach a snapshot of it to a report.

OPTIONS

`--port <n>`
: The port, 0 to 65535. Default: 0, any free port.

`--write <file>`
: Write the page instead of serving it.

EXIT STATUS

0	written, or served and stopped
1	no graph, or the file cannot be written or its folder does not exist
2	both options, a bad port, bad usage or config

EXAMPLE

```
$ slicekit visualize --write graph.html
graph.html
```

CHAPTER 10

Maintenance

The slices' state, the ratchet baselines and the session journal.

status

CI RUNS --SYNC --WRITE

NAME — **slicekit status** — list every slice's state, sync merges and regenerate BUILD.md

SYNOPSIS

```
slicekit status [--json] [--write] [--sync]
```

DESCRIPTION — Lists every slice as `<id> <state> PR <#n|-> <name>`. `--sync` first asks the git host which pull requests merged and moves those slices to `merged`. `--write` regenerates `BUILD.md` from the state files and prints `BUILD.md updated` or `unchanged` in place of the list.

WHEN TO USE — Any time, to see where slices stand. `--write` after editing slice files. `--sync` after a merge; the generated workflow runs `status --sync --write` on every merge to the default branch and commits the result.

OPTIONS

`--sync`
: Moves slices from `draft-pr` to `ready` whose pull request merged on the host to `merged`. It warns, and moves nothing, for an earlier slice whose pull request is merged.

`--write`
: Regenerates `BUILD.md`.

`--json`
: One JSON document: stage, slices, synced (with `--sync`), warnings, build (with `--write`).

EXIT STATUS

0	done
2	a bad config or slice file, a git host error, a held lock, or bad usage

EXAMPLE

```
$ slicekit status --sync
synced P1/S-02 → merged (PR #14)
P1/S-01  merged  PR #11  Health check
P1/S-02  merged  PR #14  Place an order
P1/S-03  todo  PR -  Refunds
```

state set

RARELY TYPED

NAME — **slicekit state set** — move a slice to a new state, with evidence

SYNOPSIS

```
slicekit state set <id> <to> [--commit sha] [--test-exit n] [--check-exit n]
                   [--tests a,b]
                   [--pr-number n --pr-title t --pr-url u --pr-sha s]
                   [--verdict path --round n --blockers n [--independent]]
                   [--ci-status s --ci-sha s] [--merged-at iso] [--release path]
                   [--backfill]
```

DESCRIPTION — The only writer of a slice's state file. It checks the transition's precondition against the evidence you pass and refuses otherwise. States: `todo`, `tests-red`, `building`, `draft-pr`, `in-review`, `changes-requested`, `ci-green`, `ready`, `merged`, `released`. It prints `<id> <state>`.

WHEN TO USE

The build commands call it for you. Type it to record a merge that happened outside the loop, with `merged --backfill`.

OPTIONS

`--commit <sha>`
The commit the move rests on.

`--test-exit <n>, --check-exit <n>`
Exit codes of the test command and of `check`.

`--tests <a,b>`
The test files (for `tests-red`).

`--pr-number, --pr-title, --pr-url, --pr-sha`
The pull request and its head.

`--verdict <path>, --round <n>, --blockers <n>, --independent`
A review verdict, its round, its open blockers, and whether it is independent.

`--ci-status <s>, --ci-sha <s>`
The CI result and the head it ran on.

`--merged-at <iso>`
The merge time.

`--release <path>`
The release record (for `released`).

`--backfill`
With `merged`: takes the merge time and head from the git host, for the recorded pull request or the `--pr-number` one whose title begins `[<id>]` or whose branch is the slice's. An open pull request is refused.

EXIT STATUS

0	moved
1	`refused: <reason>`, or a git host error
2	bad usage, an unknown state or slice, a held lock, or config

EXAMPLE

```
$ slicekit state set P1/S-03 merged --backfill --pr-number 17
P1/S-03  merged
```

---

state show

NAME

`slicekit state show` — print one slice's state

SYNOPSIS

`slicekit state show <id> [--json]`

DESCRIPTION

Prints one slice's state and pull request: `<id> <state> PR <#n|->`. `--json` prints the whole state file: history, frozen tests, verdicts, handoff.

WHEN TO USE

To see why a build step refuses, or what a slice has recorded.

EXIT STATUS

0	printed
2	an unknown slice, bad usage or config

EXAMPLE

```
$ slicekit state show P2/S-16
P2/S-16  merged  PR #19
```

state unfreeze

RARELY TYPED

NAME — **slicekit state unfreeze** — release one frozen test

SYNOPSIS — **slicekit state unfreeze <id> <path>**

DESCRIPTION — Releases one test frozen at `tests-red` and records it in the history. It prints `<id> unfroze <path>`.

WHEN TO USE — Seldom. `build amend` is the usual way: it also requires the slice file to record the amendment, and moves the slice back to `building` during review.

EXIT STATUS

- **0** unfrozen
- **1** the path is not a frozen test of the slice
- **2** bad usage, an unknown slice, a held lock or config

EXAMPLE

```
$ slicekit state unfreeze P1/S-02 test/order-empty.test.js
P1/S-02  unfroze test/order-empty.test.js
```

baseline update

NAME — **slicekit baseline update** — lower the ratchet baselines to today's counts

SYNOPSIS — **slicekit baseline update [--adopt] [--json]**

DESCRIPTION — Lowers the baselines of the ratchet rules (`references` and `style`) to today's counts. It refuses to raise any count: baselines only go down.

WHEN TO USE — After fixing findings, to lock in the lower count. `--adopt` once, when adding the kit to a repository that already has findings.

OPTIONS

--adopt
: A first adoption: records higher counts and prints each with its previous value. Refused when `CI` is set; a reviewer treats an adopted raise like any other finding.

--json
: Prints written, refused, adopted, rewrote and problems as JSON.

EXIT STATUS

- **0** done
- **1** a raise was refused, or a baseline file is damaged
- **2** bad usage, `--adopt` in CI, or config

EXAMPLE

```
$ slicekit baseline update
baseline style docs/spec/refunds.md: 4 → 2
```

baseline show

NAME — **slicekit baseline show** — print the ratchet baselines

SYNOPSIS — **slicekit baseline show [--json]**

DESCRIPTION — Prints the baselines as JSON, with or without `--json`. Problems with a baseline file go to stderr.

WHEN TO USE — To see what the ratchet currently allows.

EXIT STATUS

0 printed

2 bad usage

EXAMPLE

```
$ slicekit baseline show
{
  "references": {},
  "style": {}
}
```

---

journal verify

NAME

slicekit journal verify — check the session journal

SYNOPSIS

slicekit journal verify [--against <ref>]

DESCRIPTION

Checks the session journal in `.slicekit/journal/`: each file's hash chain, no unexpected entries, and no committed journal file shortened or removed since the merge base. It never writes. The session-end hook appends one line per session.

WHEN TO USE

When you suspect a journal was edited. `check`'s journal rule makes the same checks.

OPTIONS

--against <ref>

The ref whose merge base the shortening check uses. Default: the default branch.

EXIT STATUS

0 `journal: <n> lines in <m> files verified`, or `journal: no entries`

1 every problem, printed

2 bad usage

EXAMPLE

```
$ slicekit journal verify
journal: 42 lines in 6 files verified
```

---

journal show

NAME

slicekit journal show — print the last lines of the session journal

SYNOPSIS

slicekit journal show [-n <count>] [--branch <b>]

DESCRIPTION

Prints the last lines of this clone's journal file for one branch: when, who, which slice and stage, and the commits, verdicts and decisions of each session.

WHEN TO USE

To see what recent sessions did.

OPTIONS

-n <count>

How many lines. Default: 10.

--branch <b>

Which branch's file. Default: the current branch.

EXIT STATUS

0 printed, or `journal: no entries`

2 `-n` is not a positive whole number, or bad usage

EXAMPLE

```
$ slicekit journal show -n 1 --branch slice/P1-S-02-place-order
```

CHAPTER 11

Conventions

What every command shares: the global options, exit codes, how to write a value, and how this manual marks who runs a command.

Global options

`slicekit --help`
: Prints the usage of every command on standard output and exits 0, so `slicekit --help | less` shows it. Every line begins with the command's name. An unknown command prints the usage on standard error and exits 2.

`slicekit <command> --help`
: Prints that command's usage on standard output and exits 0, for example `slicekit build --help`.

`slicekit --version`
: Prints the version, for example `0.2.1`, on stdout.

Every command takes `--help`: `slicekit build --help` prints the usage of `build` and its subcommands and exits 0. Running `slicekit` with no command, or an unknown one, prints the usage of every command and exits 2.

Exit codes

Every command exits with one of three codes.

0 Success. For a gate, the thing checked is ready or clean.

1 Refused or not ready: a finding, a blocker, a state the step does not run from, or git, the git host or the tracker said no. The message says what to fix.

2 Cannot run: bad usage, an unknown ID, slice, stage or role, or a missing or invalid `.slicekit/config.json`.

Exceptions. Four commands differ on purpose: `ci bookkeeping` and `ready-label` exit 0 whatever they decide, `hook` exits 0 for every known hook and puts its decision in the JSON, and `baseline show` exits 0 even when a baseline file is damaged.

Writing values: `--option=<text>`

Write it as `--option=<text>`, with the equals sign. Otherwise the value is read as another option. This holds for every option; the procedures always use this form for free text.

```
$ slicekit decide settle D-09 --answer="-1 means no limit" --type Decision
```

Who runs a command

A command's heading carries a mark when you would seldom type it yourself. A command with no mark is one you type.

RUN BY THE AGENT
: a slash command's procedure runs it; you rarely type it

RUN BY CI
: the generated workflow runs it

RUN BY CLAUDE CODE
: a hook runs it

RARELY TYPED
: a low-level step the other commands take for you

CI RUNS …
: A variant of `RUN BY CI` that names the form the workflow uses, such as `workflow --check` or `status --sync --write`.

NOT BUILT IN 0.2.1
: Specified but not built in this version: the command prints that it is not built and exits 1.

Other conventions

- Slice IDs are `P<n>/S-<nn>`, for example `P1/S-02`. Decisions are `D-<n>`, gaps `G-<n>`, blockers `B-<n>`, should-haves `H-<n>`, contracts `C-<n>`.
- The commands own git. Agents write files and report; the CLI commits, pushes and opens pull requests. The pre-bash hook refuses git writes from a subagent.
- The git host is GitHub through the `gh` CLI. `SLICEKIT_GITHOST_FIXTURE` and `SLICEKIT_TRACKER_FIXTURE` point the kit at fixture folders instead, as the README walkthrough does.
- Where a command prints a path, it uses forward slashes on every platform.